Desenvolvimento de Sistemas Web II

HTTP e arquitetura de serviços

DIM0547 — Turma 01 · Aula 01 (Sprint 0)

Prof. Fernando · UFRN · 2026.2

Roteiro de hoje

Bloco	O que vemos
Como o curso funciona	Sprints, avaliação e o que entregar em 11/09
A Sprint 0	Visão do produto, MVP e a divisão Java × Go
HTTP em profundidade	Recurso, identificador, mensagem, semântica e erro
Arquitetura de serviços	Fronteiras, acoplamento, gRPC e falha parcial
Fechamento	Ambientes, checklist e tarefa

> O curso mudou em relação a 2026.1: deixou de ser só Go e passou a ser uma arquitetura poliglota, com Java e Go conversando por gRPC.

O combinado

Todo o material da disciplina está **público no GitHub** desde hoje.

```
github.com/fmarquesfilho/web2-2026-2
```

- Plano de curso, cronograma aula por aula, sistemática de avaliação e rúbricas
- `docs/STACK.md` com tecnologias, hospedagem gratuita e configuração do ambiente

> A avaliação da aprendizagem está detalhada em `docs/AVALIACAO.md`, e os critérios de cada entrega em `docs/RUBRICAS.md`. **Não há listas de exercícios** neste semestre.

Como o semestre funciona

Sprint 0 (4 semanas) + **3 sprints de projeto** + **bloco final**

Toda sprint segue o mesmo ciclo — a Sprint 1 como exemplo:

Quando	O quê
14 e 16/09	Aulas presenciais, com o conteúdo da sprint
21/09	Encontro online, para dúvidas do projeto
28 e 30/09	Apresentações: uma sessão online, uma em sala
02/10	**Entrega**, sexta-feira, 23:59

> A Sprint 0 foi estendida em duas semanas. Os feriados deslocam esse ritmo em cada sprint. As datas efetivas estão em `docs/CRONOGRAMA.md`.

Avaliação

Unidade	Composição	Fecha em
U1	Sprint 0 (20%) + Sprint 1 (40%) + Sprint 2 (40%)	23/10
U2	Sprint 3 (60%) + Prova (40%)	30/11
U3	Entrega final (60%) + Prova (40%)	11/12

Cada sprint: **50%** entrega técnica · **30%** atividade no repositório · **20%** comunicação

> A U1 não tem prova e fecha em 23/10, então vocês conhecem o próprio desempenho antes de decidir sobre a segunda prova. Boa parte da entrega técnica é verificável por comando, antes de entregar.

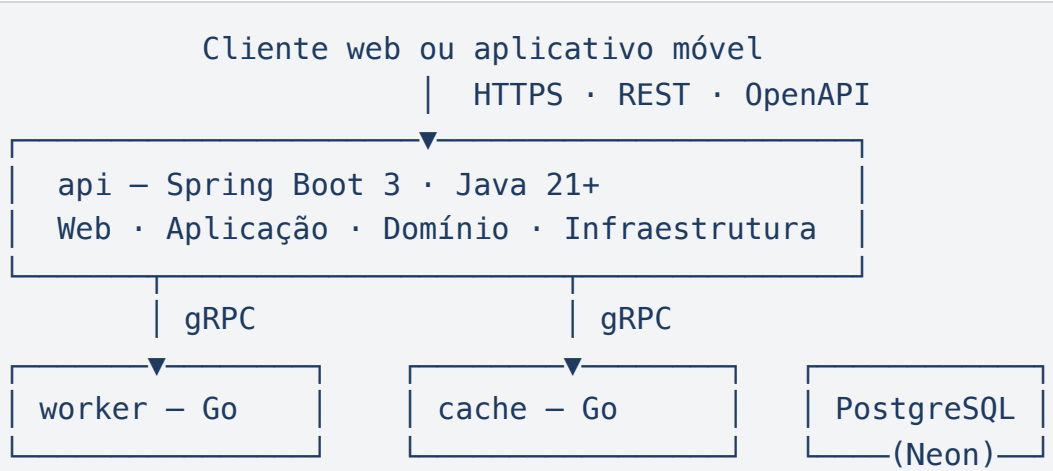
Duas provas, vale a maior

Data	Prova	Conteúdo
21/10	Prova escrita	Sprints 0 a 2
30/11	Prova de reposição, **opcional**	Cumulativa, Sprints 0 a 3

- Individuais, questões fechadas, no Multiprova, em laboratório
- Consulta permitida a **uma folha A4 manuscrita**, frente e verso
- Quem não fizer a segunda fica com a nota da primeira

> Essa mudança veio da turma: a prova estava tarde demais para dar chance de recuperação. Agora a primeira nota sai em outubro.

A arquitetura que vocês vão construir



Cada linguagem no domínio em que é mais produtiva. A justificativa dessa divisão faz parte da proposta da Sprint 0.

Datas que importam

Data	Compromisso
11/09	Entrega da Sprint 0 — monorepo, proposta e CI verde
02/10	Sprint 1 — CRUD em Spring Boot
21/10	Prova escrita, presencial, em laboratório
23/10	Sprint 2 — microsserviço Go e gRPC
20/11	Sprint 3 — Neon, cache e implantação
30/11	Prova de reposição, opcional
11/12	Entrega final

> Cronograma completo, aula por aula, em `docs/CRONOGRAMA.md`.

O que você entrega em 11/09

1. Monorepo **público** com `api/`, `services/`, `protos/`, `docs/`
2. `mise run build` e `mise run test` funcionando localmente
3. **CI verde** no GitHub Actions, compilando os dois stacks
4. `docs/proposta.md` — visão, MVP, backlog e a divisão Java × Go
5. Vídeo de 5 minutos

Grupos de **1 a 4** integrantes, formados até 11/09.

> Guia completo, com templates e exemplos, em `docs/SPRINT-0.md`.

Visão do produto

Antes de escrever código, responda por que o produto existe:

```
Para [usuários-alvo]
Que [problema ou necessidade]
O [nome do produto] é um [categoria]
Que [benefício principal]
Diferente de [alternativa existente]
Nosso produto [diferencial único]
```

> Se a equipe não consegue preencher isso, ainda não tem projeto — tem só uma ideia.

Divisão do trabalho entre Java e Go

a decisão técnica central da proposta

Vai para o serviço Java	Vai para um microsserviço Go
Entidades de domínio e regras	Coleta e integração com sistemas externos
Persistência e migrações	Processamento em lote ou agendado
Autenticação e autorização	Trabalho concorrente de I/O intensivo
Orquestração dos casos de uso	Cache e pré-computação

> O critério **não** é "qual linguagem eu prefiro". É qual característica do trabalho justifica cada escolha.

HTTP: o protocolo que sustenta tudo

Toda API web é, no fundo, um acordo sobre **como interpretar mensagens de texto** trocadas por uma conexão.

```
GET /produtos/42 HTTP/1.1
Host: api.exemplo.br
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: max-age=300

{"id": 42, "nome": "Café"}
```

> Requisição e resposta. É essa simplicidade que permitiu a web escalar.

Três palavras que não são sinônimas

Termo	Definição
Recurso	Qualquer coisa que se possa nomear e referenciar: o produto 42, a lista de pedidos de hoje, o relatório de vendas de agosto
Identificador	A URI que aponta para o recurso — estável, mesmo que o conteúdo mude
Representação	Uma forma concreta do recurso num instante: JSON, XML, CSV, HTML, uma imagem

O cliente **nunca** manipula o recurso. Ele manipula representações que o servidor entrega e recebe.

> Por isso a mesma URI pode devolver JSON para o aplicativo e HTML para o navegador, sem deixar de ser o mesmo recurso.

Anatomia de uma URI

```
https://api.exemplo.br:443/v1/produtos/42?campos=nome,preco#detalhe
└──┘   └──────────────┘└─┘└──────────┘└──────────────────┘└────┘
esquema    autoridade   porta   caminho         consulta   fragmento
```

- **Caminho** identifica o recurso — use substantivos, no plural, sem verbos
- **Consulta** parametriza a representação: filtro, ordenação, paginação, campos
- **Fragmento** nunca chega ao servidor; é resolvido pelo cliente

> `/produtos/42` é recurso. `/buscarProduto?id=42` é chamada de procedimento disfarçada de URL. A diferença aparece no cache, nos logs e na documentação.

O que acontece antes do primeiro byte

```
1. DNS      api.exemplo.br  ⟶  200.130.x.x
2. TCP      handshake de 3 vias
3. TLS      handshake, certificado, chaves de sessão
4. HTTP     requisição ⟶ resposta
```

Cada etapa custa pelo menos uma ida e volta na rede. Da UFRN a um servidor na Virgínia, cada ida e volta são dezenas de milissegundos.

> É por isso que **conexão persistente** e cache importam tanto: eles eliminam etapas inteiras, não apenas bytes. Otimizar o SQL de um endpoint que poderia nem ter sido chamado é otimizar a coisa errada.

HTTP/1.1, HTTP/2 e HTTP/3

Versão	Transporte	Característica
1.1	TCP	Texto puro; conexão persistente; uma resposta por vez em cada conexão
2	TCP	Binário, em quadros; **multiplexação** de várias trocas na mesma conexão; cabeçalhos comprimidos
3	QUIC sobre UDP	Multiplexação sem o bloqueio de fila do TCP; handshake mais curto

A **semântica é a mesma nas três**: métodos, status e cabeçalhos não mudam.

> Guardem "binário e multiplexado": é exatamente sobre HTTP/2 que o gRPC é construído, no fim da aula.

Sem estado, e o que isso custa

Sem estado: o servidor não guarda contexto entre requisições. Toda requisição carrega tudo o que é preciso para ser entendida sozinha.

Ganha

- Qualquer réplica atende qualquer requisição
- Escala horizontal trivial
- Falha de instância não perde sessão

Paga

- Credencial repetida a cada chamada
- Mensagens maiores
- Estado tem de morar em algum lugar

> Onde mora o estado é uma decisão de projeto: token assinado no cliente, sessão no Redis, ou no banco. **"Não tem estado" nunca significa "o estado sumiu".**

Anatomia da mensagem

```
POST /pedidos HTTP/1.1          ← linha inicial: método, alvo, versão
Host: api.exemplo.br            ┐
Content-Type: application/json  │ cabeçalhos, um por linha
Content-Length: 38              ┘
                                ← linha em branco: fim dos cabeçalhos
{"produtoId": 42, "qtd": 2}     ← corpo
```

Na resposta, a linha inicial vira `HTTP/1.1 201 Created`.

> A linha em branco não é estética: é o delimitador do protocolo. Compreender que a mensagem é **texto estruturado** é o que permite depurar com `curl -v` em vez de adivinhar.

Métodos: o verbo carrega semântica

Método	Semântica	Seguro	Idempotente
`GET`	Ler um recurso	sim	sim
`HEAD`	Ler só os cabeçalhos	sim	sim
`POST`	Criar, ou operação não idempotente	não	não
`PUT`	Substituir por completo	não	sim
`PATCH`	Modificar parcialmente	não	não
`DELETE`	Remover	não	sim

Seguro = não altera estado observável. **Idempotente** = repetir *N* vezes tem o mesmo efeito de fazer uma vez.

Por que idempotência não é teoria

O cliente envia `POST /pedidos`. A resposta se perde na rede. E agora?

```
cliente ──POST──► servidor    (pedido criado)
cliente ◄──✗─────  servidor    (resposta perdida)
cliente: repito ou não?
```

Se fosse `PUT /pedidos/{id-que-eu-escolhi}`, repetir seria seguro. Como é `POST`, repetir cria dois pedidos.

Solução usual: cabeçalho `Idempotency-Key` com um identificador gerado pelo cliente; o servidor guarda a resposta e devolve a mesma se a chave se repetir.

> Bibliotecas de retry, proxies e filas **repetem sozinhas**. Um método marcado como idempotente é uma promessa que alguém já está usando.

PUT ou PATCH

PUT — substituição total

```
PUT /produtos/42
Content-Type: application/json

{"nome": "Café", "preco": 25.0,
 "ativo": true}
```

Campo omitido é campo apagado.

PATCH — alteração parcial

```
PATCH /produtos/42
Content-Type: application/merge-patch+json

{"preco": 27.5}
```

`null` significa "remova este campo".

> `PATCH` sem declarar o formato deixa o cliente adivinhando. Escolham um — *merge patch*, RFC 7386, atende a maioria dos casos — e documentem.

Status: as cinco famílias

`1xx` informativo

`100` Continue · `101` Switching

`2xx` deu certo

`200` OK · `201` Created + `Location`

`202` Accepted · `204` No Content

`3xx` redirecionamento

`301` permanente · `304` Not Modified

`4xx` erro do cliente

`400` `401` `403` `404`

`409` `412` `422` `429`

`5xx` erro do servidor

`500` `502` `503` `504`

> A regra de ouro: **4xx significa "não adianta repetir do jeito que você mandou"**; 5xx significa "pode ser que repetir funcione". Clientes automatizados decidem retry com base nisso.

Códigos que costumam ser confundidos

Confusão	Quem é quem
`401` × `403`	401 = "não sei quem você é" · 403 = "sei, e você não pode"
`400` × `422`	400 = não consegui nem interpretar · 422 = interpretei, mas viola regra de negócio
`404` × `409`	404 = não existe · 409 = existe e conflita com o estado atual
`429`	Excesso de requisições; **sempre** acompanhado de `Retry-After`

> Um caso à parte: devolver `200 OK` com `{"erro": "..."}` no corpo. Isso quebra todo cliente, proxy e monitoramento que confia no status.

Erro que o cliente consegue tratar

Status sozinho não basta: `422` não diz *qual* regra falhou. A RFC 9457 padroniza o corpo:

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{ "type": "https://api.exemplo.br/erros/estoque-insuficiente",
  "title": "Estoque insuficiente",
  "status": 422,
  "detail": "Produto 42 tem 3 unidades; foram pedidas 10.",
  "instance": "/pedidos/9f2c" }
```

> `type` é o identificador estável que o cliente compara — nunca a mensagem em português. É o formato adotado no projeto, a partir da Sprint 1.

Cabeçalhos que mudam o comportamento

Cabeçalho	Para quê
`Content-Type`	Como interpretar o corpo desta mensagem
`Accept`	Que formatos o cliente aceita na resposta
`Authorization`	Credencial da requisição
`Cache-Control`	Se, por quem e por quanto tempo pode ser guardado
`ETag` / `If-None-Match`	Revalidação: mudou desde a última vez?
`Location`	Onde está o recurso criado, ou para onde redirecionar

> Cabeçalhos de **requisição** pedem; de **resposta** informam; de **representação** descrevem o corpo. Confundir os três é o que produz APIs em que `Content-Type` aparece onde deveria estar `Accept`.

Negociação de conteúdo e versionamento

```
Accept: application/json, application/xml;q=0.8
Accept-Language: pt-BR, en;q=0.5
Accept-Encoding: gzip, br
```

O `q` é um peso de preferência, de 0 a 1. O servidor escolhe e responde com `Vary` indicando o que influenciou a escolha.

Versionar a API usa o mesmo raciocínio, com três alternativas:

Onde	Exemplo	Custo
Caminho	`/v1/produtos`	Simples, visível; duplica URIs
Cabeçalho	`Accept: ...;version=1`	URI estável; invisível no navegador
Campo	`{"apiVersion": 1}`	Não interage com cache nem com proxy

Arquitetura de serviços

A pergunta não é "microsserviços ou não". É **onde traçar as fronteiras** do sistema, e por quê.

Arquitetura é o conjunto de decisões que **custam caro para reverter**: fronteiras entre partes, forma de comunicação, onde os dados moram.

Três respostas, em ordem de acoplamento:

Monólito · **Monólito modular** · **Microsserviços**

> Cada uma resolve um problema e cria outro. A escolha errada custa caro nos dois sentidos.

Acoplamento e coesão

Conceito	Definição	Sinal de que está ruim
Acoplamento	O quanto uma parte depende de detalhes internos de outra	Mudar um campo obriga a mexer em quatro módulos
Coesão	O quanto o que está junto muda pela mesma razão	Toda mudança de negócio espalha por três serviços

O objetivo é **acoplamento fraco e coesão forte** — em qualquer das três arquiteturas.

> Microsserviços não reduzem acoplamento: eles **transformam** acoplamento de código em acoplamento de rede, que é mais difícil de enxergar e mais caro de errar.

Monólito

Um processo, um deploy, um banco.

A favor

- Simples de rodar e depurar
- Refatoração atravessa tudo
- Transação única no banco
- Sem latência de rede interna

Contra

- Acoplamento cresce sem barreira
- Um deploy para tudo
- Escala em bloco
- Uma linguagem só

> É o ponto de partida certo para quase todo projeto novo. O erro comum não é começar monolítico — é **nunca revisar** essa decisão.

Monólito modular

Um deploy, mas com **fronteiras internas explícitas** e verificadas.

```
api/
 ├── catalogo/     ← módulo, com sua própria camada de domínio
 ├── pedidos/      ← só conversa com catalogo por interface pública
 └── shared/
```

As fronteiras são garantidas por ferramenta, não por disciplina: no Java, **ArchUnit** falha o build se um módulo importar o interno de outro; em Go, o **arch-go** faz o equivalente.

> É o meio-termo subestimado. Você ganha a disciplina dos microsserviços sem pagar a conta de rede, e prepara o terreno para extrair um serviço quando fizer sentido.

Microsserviços

Serviços independentes, com deploy e dados próprios, conversando por rede.

Ganha

- Deploy e escala independentes
- Linguagem por serviço
- Falha isolada
- Times autônomos

Paga

- Rede: latência e falha parcial
- Consistência eventual
- Observabilidade obrigatória
- Contratos para versionar

> A conta que se esquece de somar: **operação**. Cinco serviços são cinco pipelines, cinco deploys e cinco fontes de log para correlacionar.

Como decidir a fronteira

Um serviço separado se justifica quando há uma **razão** para separá-lo:

Razão	Exemplo
Perfil de carga diferente	Coleta massiva vs. CRUD interativo
Ciclo de vida diferente	Muda toda semana vs. estável há um ano
Requisito técnico distinto	Concorrência pesada, processamento de mídia
Fronteira de domínio clara	Cobrança e catálogo não compartilham dados

A última linha tem nome: **contexto delimitado** — a região em que um termo do negócio tem um significado só. "Pedido" no catálogo e "pedido" na cobrança raramente são a mesma coisa.

> **"Queríamos usar Go" não é uma razão.** É esse raciocínio que a proposta de vocês precisa apresentar.

Síncrono ou assíncrono

Forma	Quando serve	Custo
REST sobre HTTP	Fronteira externa, clientes diversos, cacheável	Texto verboso; acoplamento temporal
gRPC	Chamada interna entre serviços, contrato forte	Não é amigável ao navegador
Mensageria	Trabalho que pode esperar; produtor não conhece consumidor	Entrega ao menos uma vez; ordem não garantida

Acoplamento temporal: em chamada síncrona, se o outro serviço está fora do ar, você está fora do ar.

> No projeto de vocês, a fronteira externa é REST e a interna é gRPC. Mensageria é opcional — e quem usar precisa explicar o que ganhou com ela.

gRPC

Chamada de procedimento remoto sobre **HTTP/2**, com corpo em **Protocol Buffers** — binário e tipado.

```
service Coletor {
  rpc Coletar(ColetarRequest) returns (ColetarResponse);
  rpc Acompanhar(AcompanharRequest) returns (stream Evento);
}
```

- Quatro formas: unária, fluxo do servidor, fluxo do cliente, bidirecional
- O `.proto` **gera** o cliente Java e o servidor Go: nada de escrever cliente HTTP à mão
- Traz prazo por chamada, cancelamento e códigos de status próprios

> A vantagem que mais importa aqui não é desempenho: é o **contrato verificável**. O compilador reprova o que num JSON só quebraria em produção.

O contrato entre serviços

Se dois serviços conversam, existe um contrato. A escolha é se ele é **explícito e verificado** ou implícito e frágil.

Ferramenta	O que garante
`buf lint`	O `.proto` segue a convenção adotada
`buf breaking`	A mudança não quebra quem já usa o contrato
ArchUnit / arch-go	Nenhum módulo atravessa a fronteira interna

Compatível para trás: **acrescentar** campo opcional, sim; renomear, reusar número de campo ou mudar tipo, não.

> Quebrou compatibilidade? O **CI reprova antes do merge**. É a mesma ideia do ArchUnit, aplicada à fronteira entre serviços em vez de entre pacotes.

Falha parcial: o problema que a rede cria

Em um monólito, uma chamada de método falha ou retorna. Na rede, existe um terceiro caso: **não sei o que aconteceu**.

Mecanismo	O que faz
Prazo	Toda chamada morre em *N* ms; sem isso, uma lentidão vira fila infinita
Retry com recuo	Repete espaçando as tentativas, com variação aleatória
Disjuntor	Após *N* falhas, para de tentar por um tempo — fechado, aberto, meio-aberto
Anteparo	Limita quantas chamadas simultâneas cada dependência pode consumir

> Retry **sem** prazo e **sem** recuo não salva o sistema: derruba o serviço lento de vez. Repetir só é seguro se a operação for idempotente — a ideia da primeira parte da aula, agora na fronteira interna.

Se não dá para observar, não dá para operar

Sinal	Responde
Log	O que aconteceu nesta requisição
Métrica	Quantas vezes, quão rápido, com que taxa de erro
Rastro	Por quais serviços esta requisição passou, e quanto custou em cada um

O elo entre os três é o **identificador de correlação**, propagado de serviço em serviço.

> Com um serviço, o log basta. Com três, sem correlação você tem três diários que não conversam — e nenhuma resposta para "por que ficou lento".

Onde escrever código

Ambiente	Cadastro	Serve para
Máquina local	—	Tudo, com Docker Desktop
GitHub Codespaces	sim	VS Code no navegador, Java, Go, Docker, testes
Google Cloud Shell	conta Google	Compilar e rodar testes, 5 GB persistentes

Estudantes têm **180 h por mês** de Codespaces com o GitHub Student Pack.

> Recomendo versionar um `.devcontainer/devcontainer.json` no monorepo. Isso resolve o "na minha máquina funciona" de vez — e é o mesmo princípio do `docker-compose.yml`.

Para segunda, 24/08

Tarefas

- Ir organizando para formar o grupo do projeto, com 1 a 4 integrantes (se quiser pode usar o canal do Discord pra achar colegas)
- Trazer uma ideia de produto com um recorte de domínio

Recomendado

- Instalar `mise` · `mise.jdx.dev` e o Docker Desktop
- Ler `docs/STACK.md` do repositório da disciplina

Referências da aula

HTTP

- MDN — *HTTP* · `developer.mozilla.org/docs/Web/HTTP`
- RFC 9110 *Semantics* · RFC 9111 *Caching* · RFC 9457 *Problem Details*

Arquitetura

- Newman, *Building Microservices*, 2ª ed. · Fowler, *MonolithFirst*
- Fielding, *Architectural Styles*, cap. 5 · Richardson Maturity Model

Disciplina

- `github.com/fmarquesfilho/web2-2026-2`